FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS
DEPARTAMENTO DE TECNOLOGÍA INFORMÁTICA

Algoritmos y Estructuras de Datos II

Trabajo Práctico 1 – TDAs: Conceptos Básicos

ENUNCIADO

MIEMBROS:

APELLIDO	NOMBRE
TOMAS	TORRES
AGUSTIN	LOPEZ
RENZO	GARABANO
IVAN	CANCINOS
MATIAS	ESTECHE

Definición, implementación y utilización

En este trabajo van a definir, implementar y utilizar tres TDAs clásicos: Pila, Cola y Cola con Prioridad, usando implementaciones estáticas (basadas en arreglos). El objetivo es consolidar la diferencia entre especificación (qué hace el TDA) e implementación (cómo lo hace), y practicar la resolución de problemas usando exclusivamente las operaciones de la interfaz, sin asumir ningún detalle interno de la implementación.

Condiciones de entrega

Trabajo individual. Lenguaje: Java. Cada método pedido debe resolverse usando únicamente las operaciones del TDA correspondiente (crear, apilar/desapilar, encolar/desencolar, insertar/extraer, esVacia, etc.) — no accedan directamente al arreglo interno ni al índice de tope/frente desde fuera de la implementación. Se evalúa tanto la corrección como la complejidad temporal de cada solución.

Parte 1 — TDA Pila

1.1 Especificación

Escriban la especificación del TDA Pila siguiendo el formato visto en clase: dominio y operaciones, cada una con su signatura y sus condiciones de precondición/postcondición. Como mínimo debe incluir: crear, apilar, desapilar, tope, esVacia.

1.2 Implementación

Implementen el TDA Pila de forma estática (con un arreglo de tamaño fijo) de dos maneras distintas:

- **Variante A:** el tope de la pila se mantiene en el último lugar ocupado del arreglo.
- **Variante B:** el tope de la pila se mantiene siempre en la posición 0 del arreglo.

Para cada variante, indiquen la complejidad temporal de apilar, desapilar y tope. Con esa información, respondan y justifiquen: ¿cuál de las dos variantes conviene usar? ¿Depende de algo?

R: Conviene usar la Pila con tope al final del array entonces entendemos que no requieren recorrer ni mover elementos. En cambio, con tope al principio cada apilar/desapilar obliga a correr todos los elementos existentes, lo cual es O(n). Por lo que conviene usar la variante A.

1.3 Utilización

Usando únicamente las operaciones del TDA Pila (sin importar cuál de las dos variantes uses — el código debe funcionar igual con cualquier implementación), resuelvan:

1. **pasarPila(origen: Pila): Pila —** devuelve una nueva pila con todos los elementos de origen, dejando origen vacía.
2. **copiarPila(p: Pila): Pila —** devuelve una copia de p, dejando p en su estado original (mismo contenido y mismo orden que antes de llamar al método).
3. **invertirPila(p: Pila): Pila —** devuelve una nueva pila con los elementos de p en el orden inverso. Debe resolverse en forma recursiva.
4. **masDeUnaOcurrencia(p: Pila): boolean —** indica si algún elemento aparece más de una vez en p. Al finalizar, p debe quedar con su contenido y orden original.
5. **eliminarImpares(p: Pila): Pila —** devuelve una nueva pila con los elementos de p que son pares, en el mismo orden relativo en que estaban en p.

Parte 2 — TDA Cola

2.1 Especificación

Escriban la especificación del TDA Cola: dominio y operaciones (crear, encolar, desencolar, frente, esVacia), con sus pre/postcondiciones.

2.2 Implementación

Implementen el TDA Cola de forma estática (con un arreglo de tamaño fijo) de dos maneras distintas:

- **Variante A — Cola lineal simple:** frente y fin avanzan siempre hacia adelante en el arreglo (sin reutilizar los lugares que quedan libres al desencolar).
- **Variante B — Cola circular:** frente y fin avanzan usando aritmética modular, reutilizando los lugares liberados.

Analicen la complejidad temporal de encolar y desencolar en cada variante, y justifiquen cuál conviene usar en la práctica.

R: La cola circular es más eficiente porque evita el desperdicio de espacio que ocurre en la cola lineal. En la cola lineal, cuando se desencola un elemento, la posición que queda libre al principio no se reutiliza. En cambio, en la circular, al llegar al final del arreglo se vuelve al principio y se reutilizan esas posiciones.

En cuanto a complejidad, tanto encolar como desencolar son O(1) en ambas implementaciones, porque no hay que recorrer la cola para encontrar una posición. Se accede directamente mediante los índices frente/inicio y fin. Por eso, el costo se mantiene constante independientemente de cuántos elementos tenga la cola.

2.3 Utilización

Usando únicamente las operaciones del TDA Cola, resuelvan:

6. **pasarCola(origen: Cola): Cola —** devuelve una nueva cola con todos los elementos de origen, en el mismo orden, dejando origen vacía.
7. **invertirColaConPila(c: Cola): Cola —** devuelve una nueva cola con los elementos de c en orden inverso, usando una Pila auxiliar.
8. **invertirColaSinPila(c: Cola): Cola —** resuelve el mismo problema que el punto anterior, pero sin usar ninguna estructura auxiliar (ni Pila, ni arreglo, ni Cola extra): la inversión debe lograrse únicamente con recursión pura sobre la propia Cola.
9. **finalCoincide(c1: Cola, c2: Cola, k: entero): boolean —** indica si los últimos k elementos de c1 coinciden, en el mismo orden, con los últimos k elementos de c2. Ambas colas deben quedar con su contenido y orden original al finalizar.

Parte 3 — TDA Cola con Prioridad

3.1 Especificación

Escriban la especificación del TDA Cola con Prioridad: dominio y operaciones (crear, insertar — que recibe un elemento y una prioridad —, extraerMax, verMax, esVacia), con sus pre/postcondiciones. Definan también qué hace su TDA cuando dos elementos tienen la misma prioridad (criterio de desempate).

3.2 Implementación

Implementen el TDA Cola con Prioridad de forma estática (con un arreglo de tamaño fijo) de dos maneras distintas:

- **Variante A — Desordenada:** los elementos se insertan en el primer lugar libre, sin mantener ningún orden; extraerMax debe recorrer el arreglo para encontrar el máximo.
- **Variante B — Ordenada:** los elementos se mantienen siempre ordenados por prioridad dentro del arreglo, de forma que extraerMax accede directamente al máximo.

Para cada variante, dado que ambas deben respetar el criterio de desempate definido en 3.1, analicen la complejidad temporal de insertar, extraerMax y verMax. Con esos datos, discutan: ¿existe una variante que sea mejor que la otra en todos los casos, o la elección depende del contexto de uso? Justifiquen.

R:

Complejidad temporal

Operación	Desordenada	Ordenada	Motivo principal
insertar	**O(1)**	**O(n)**	La ordenada puede tener que correr todos los elementos.
extraerMax	**O(n)**	**O(1)**	La desordenada debe buscar el máximo. La ordenada lo tiene al final.
verMax	**O(n)**	**O(1)**	La desordenada busca el máximo y la ordenada accede a la última posición.

Elección de la variante

No hay una variante mejor en todos los casos. Si el programa inserta muchos elementos y casi no extrae, conviene la desordenada porque insertar cuesta O(1). Si consulta o extrae el máximo muchas veces, conviene la ordenada porque esas operaciones cuestan O(1). La elección depende de cuál operación se usa más.

3.3 Utilización

Usando únicamente las operaciones del TDA Cola con Prioridad, resuelvan:

10. **combinar(cp1: ColaPrioridad, cp2: ColaPrioridad): ColaPrioridad —** devuelve una nueva cola con prioridad que contiene todos los elementos de cp1 y cp2, respetando el criterio de desempate definido en 3.1 (es decir, el orden relativo de inserción entre elementos de igual prioridad debe conservarse). Ambas colas de entrada deben quedar vacías al finalizar.

11. **invertirColaConColaPrioridad(c: Cola): Cola —** devuelve una nueva cola con los elementos de c en orden inverso, usando una ColaPrioridad auxiliar (pensar qué prioridad asignarle a cada elemento a medida que se recorre c).

12. **sumarValoresPrioridadPar(cp: ColaPrioridad): entero —** devuelve la suma de los valores de todos los elementos cuya prioridad es un número par. cp debe quedar con su contenido y orden original al finalizar.

Entrega

Entreguen el código fuente completo (una clase por TDA y por variante, más una clase con los métodos de utilización de cada parte) junto con un breve informe (puede ser el mismo Word/PDF con las respuestas a las consignas de análisis) que incluya:

- Las especificaciones pedidas en 1.1, 2.1 y 3.1.
- Las tablas de complejidad temporal y las justificaciones pedidas en 1.2, 2.2 y 3.2.
- Para cada método de las secciones de utilización, la complejidad temporal de la solución presentada.